

Вопросы по курсу "Параллельное программирование"

1 Лекция 1: introduction to multithreading

Ключевые понятия: concurrency, parallelism, agents, threads, scheduler, threading models, interleaving execution, Amdahl's law, race condition, data race, deadlock, wait-for graph

- 1.1 Конкурентность (concurrency) и параллелизм (parallelism). Системы разделения времени (time-sharing), многозадачность (multitasking). Процессы и потоки. Вытесняющая многозадачность и кооперативная многозадачность. Планировщик ОС: квант планирования, смена контекста, метрики эффективности планирования, стратегии планирования, алгоритм round-robin. Задачи, легко поддающиеся параллелизации (embarrassingly parallel problems). Закон Амдала.
- 1.2 Способы выделения вычислительных ресурсов для задач (threading models): 1:1, N:1, N:M. Модель исполнения, основанная на переборе возможных смен контекста на одном исполнителе (interleaving model). Трасса исполнения многопоточной программы (concurrent execution trace). Недетерминизм многопоточного исполнения, состояние гонки (race condition), гонка данных (data race). Видимость (visibility). Wait-for graph. Состояние тупика (deadlock). Инверсия приоритетов.

2 Лекция 2: mutual exclusion

Ключевые понятия: thread safety, concurrent consistency, mutual exclusion, critical section, deadlock-freedom, starvation, fairness, check-then-act, reentrancy, admission policy, code locking, data locking, lock splitting, lock ordering, dining philosophers problem

- 2.1 Неформальный подход к потоковой безопасности (thread-safety), примеры.

Взаимное исключение. Критическая секция (mutex). Алгоритм взаимного исключения для двух потоков, основанный на флагах: доказательство свойства mutual exclusion, доказательство свойства deadlock freedom. Голодание (starvation). Реентерабельность (reentrancy). Admission policy, связь с честностью (fairness), наблюдаемым временем отклика (latency) и производительностью (throughput).

Задача об обедающих философах (dining philosophers problem).

- 2.2 Неформальный подход к потоковой безопасности (thread-safety), примеры.

Взаимное исключение. Критическая секция (mutex). Голодание (starvation). Реентерабельность (reentrancy). Admission policy, связь с честностью (fairness), наблюдаемым временем отклика (latency) и производительностью (throughput).

Блокировка кода (code locking), блокировка данных (data locking). Расщепление мьютексов (lock splitting). Упорядочивание критических секций для предотвращения состояния тупика (lock ordering, deadlock prevention).

Задача об обедающих философах (dining philosophers problem).

3 Лекция 3: signalling

Ключевые понятия: thread factory, executor, future, thread pooling, backoff policy, signalling, condition variable, lost signal, spurious wakeup, wait set admission policy, blocking queue, predicate invalidation, thread-starvation deadlock, resource deadlock

3.1 Неформальный подход к потоковой безопасности (thread-safety), примеры.

Передача сигнала от одного потока другому (signalling). Активное ожидание с переключением контекста (backoff policies). Условная переменная (condition variable).

Thread pools: Runnable, ThreadFactory, Executor, Callable, Future, ExecutorService, fixedThreadPool, singleThreadExecutor, cachedThreadPool.

Проблемы, характерные для пулов потоков: взаимные блокировки, утечки памяти, голодание, искажение владения (ownership, thread-local).

3.2 Неформальный подход к потоковой безопасности (thread-safety), примеры.

Передача сигнала от одного потока другому (signalling). Активное ожидание с переключением контекста (backoff policies). Условная переменная (condition variable).

Пример API: wait/notify/notifyAll. Пример API: await/signal/signalAll. Проблема потерянного сигнала (lost signal), внезапного пробуждения (spurious wakeup), устаревания условия (predicate invalidation). Честность (fairness) и голодание (starvation). Особенности использования нескольких условных переменных, ассоциированных с одним и тем же мьютексом, пример очереди с ограниченной вместимостью (bounded buffer).

4 Лекция 4: advanced synchronization primitives

Ключевые понятия: monitor, synchronized, completion service, thundering herd problem, wait morphing, countdown latch, semaphore, read-write lock, thread local, replication, snapshotting

4.1 Взаимное исключение. Критическая секция (mutex). Передача сигнала от одного потока другому (signalling). Активное ожидание с переключением контекста (backoff policies). Условная переменная (condition variable).

Мониторы в языке Java (built-in monitor). Описание API. Ключевое слово **synchronized**. Admission policy. Реентерабельность (reentrancy), честность (fairness), владение (ownership). Пример CompletionService. Проблема грохочущего стада (thundering herd). Оптимизация wait morphing.

4.2 Взаимное исключение. Критическая секция (mutex). Передача сигнала от одного потока другому (signalling). Активное ожидание с переключением контекста (backoff policies). Условная переменная (condition variable).

CountDownLatch. На примере этой структуры данных проиллюстрируйте следующие многопоточные дефекты: проблема потерянного сигнала (lost signal), устаревание условия (predicate invalidation), внезапное пробуждение (spurious wakeup), грохочущее стадо (thundering herd), инверсия приоритета (priority inversion), голодание (starvation), состояние тупика (deadlock). Возможно вам понадобится разъяснить отсутствие перечисленных проблем в структуре данных.

4.3 Взаимное исключение. Критическая секция (mutex). Передача сигнала от одного потока другому (signalling). Активное ожидание с переключением контекста (backoff policies). Условная переменная (condition variable).

Semaphore. На примере этой структуры данных проиллюстрируйте следующие многопоточные дефекты: проблема потерянного сигнала (lost signal), устаревание условия (predicate invalidation), внезапное пробуждение (spurious wakeup), грохочущее стадо (thundering herd), инверсия приоритета (priority inversion), голодание (starvation), состояние тупика (deadlock). Возможно вам понадобится разъяснить отсутствие перечисленных проблем в структуре данных.

4.4 Взаимное исключение. Критическая секция (mutex). Передача сигнала от одного потока другому (signalling). Активное ожидание с переключением контекста (backoff policies). Голодание (starvation). Реентерабельность (reentrancy). Admission policy, связь с честностью (fairness), наблюдаемым временем отклика (latency) и производительностью (throughput).

ReadWriteLock. Предпочтение читающего или пишущего потока. Усиление (read-to-write promotion) и ослабление (write-to-read downgrade). На примере этой структуры данных проиллюстрируйте следующие многопоточные дефекты: проблема потерянного сигнала (lost signal), устаревание условия (predicate invalidation), внезапное пробуждение (spurious wakeup), грохочущее стадо (thundering herd), инверсия приоритета (priority inversion), голодание (starvation), состояние тупика (deadlock). Возможно вам понадобится разъяснить отсутствие перечисленных проблем в структуре данных.

4.5 Поточно-локальные (поточно-специфичные, thread-local) данные. Паттерн "репликация" (replication), паттерн "снимок состояния" (snapshotting). Алгоритм распределенного счетчика, ориентированный на масштабируемость операции `increment`.

5 Лекция 5: cancellation and interruption

Ключевые понятия: concurrent state machine, forced cancellation, cooperative cancellation, cancellation token, interruption, timeouts

5.1 Отмена задач (cancellation).

Thread pools: `Runnable`, `ThreadFactory`, `Executor`, `Callable`, `Future`, `ExecutorService`, `fixedThreadPool`, `singleThreadExecutor`, `cachedThreadPool`.

Техники декомпозиции многопоточных программ: конечный автомат для интерфейса `Future`, отмена задач (cancellation of not-yet-processed tasks). Односторонняя отмена задач (forced cancellation), преимущества и недостатки. Кооперативная отмена задач с помощью cancellation token.

5.2 Прерывание задач (interruption). Прерывание потоков (thread interruption) с помощью архитектуры thread-specific cancellation token. Особенности поддержки такого решения на уровне языка программирования и его стандартной библиотеки. Прерываемые (interruptible) и непрерываемые (uninterruptible) методы. Прерывание задач (task interruption). Политики распространения запроса на прерывания. Упорядочение асинхронных событий: прерывание (interrupt), сигнал условной переменной (signal/notify), срабатывания таймера (timeout).

6 Лекция 6: reliability

Ключевые понятия: documenting protocols and classes, concurrent invariants, assertions, stress testing, estimating bug probability, static and dynamic checks, scheduling randomization and determinisation, model checking

6.1 Документирование многопоточных инвариантов. Проверка инвариантов потоковой безопасности для различных структур данных. Assertions. Препятствия для написания надежных многопоточных программ.

Тестирование многопоточных программ. Юнит-тестирование, стресс-тестирование. Анализ вероятности негативного сценария, оценка требуемых вычислительных ресурсов для тестирования.

6.2 Документирование многопоточных инвариантов. Проверка инвариантов потоковой безопасности для различных структур данных. Assertions. Препятствия для написания надежных многопоточных программ.

Инструменты автоматического анализа многопоточных программ. Статический анализ и поиск дефектов. Проверки времени исполнения. Контроль порядка исполнения конкурентной программы (scheduling control): с помощью тестов (mocking, inheritance), с помощью метаязыковых средств (aspects, bytecode transformation), с помощью операционной системы (affinity control).

6.3 Раздел про проверку моделей (model checking) не выносится на экзамен 2026 года.

7 Лекция 7: theoretical foundations of concurrency

Ключевые понятия: timeline, events, precedence, 2-thread mutual exclusion, deadlock freedom, starvation freedom, N-thread mutual exclusion, sequential objects and specifications, concurrent objects, linearizability

7.1 Формализация многопоточного исполнения: линия времени, атомарное событие (event), трасса исполнения потока (trace model), временной интервал. Частичный порядок, отношение предшествования (precedence), его свойства.

Формальное определение взаимного исключения (mutual exclusion), отсутствия тупика (deadlock freedom), отсутствия голодания (starvation freedom).

Алгоритмы **LockOne**, **LockTwo**, алгоритм Петерсона, доказательства их основных свойств.

Теорема о нижней границе: сколько необходимо независимых ячеек памяти, чтобы добиться взаимного исключения N потоков.

7.2 Формализация многопоточного исполнения: линия времени, атомарное событие (event), трасса исполнения потока (trace model), временной интервал. Частичный порядок, отношение предшествования (precedence), его свойства.

Формальное определение взаимного исключения (mutual exclusion), отсутствия тупика (deadlock freedom), отсутствия голодания (starvation freedom).

Алгоритм **FilterLock**, доказательства его основных свойств.

Теорема о нижней границе: сколько необходимо независимых ячеек памяти, чтобы добиться взаимного исключения N потоков.

7.3 Формализация многопоточного исполнения: линия времени, атомарное событие (event), трасса исполнения потока (trace model), временной интервал. Частичный порядок, отношение предшествования (precedence), его свойства.

Последовательный объект (sequential object), последовательная спецификация (sequential specification). Понятие о многопоточной согласованности (concurrent consistency). Линеаризуемое исполнение (linearizable execution), линеаризуемый объект (linearizable object), точка линеаризации (linearization point). Примеры.

8 Лекция 8: register constructions

Ключевые понятия: obstruction-free, lock-free, wait-free, safe register, regular register, atomic register, register snapshot

8.1 Условия прогресса (progress conditions). Dependent blocking progress conditions. Примеры. Non-blocking progress conditions. Примеры. Dependent non-blocking progress conditions. Пример. Теорема о связи условий прогресса между собой.

Регистр (register): емкость (capacity), поддерживаемая конкурентность (concurrency), гарантии консистентности (consistency). Пространство регистров. Wait-free реализация объекта.

Теоремы о выразительной силе:

- SRSW Safe Boolean $\rightarrow$ MRSW Safe Boolean
- MRSW Safe Boolean $\rightarrow$ MRSW Regular Boolean
- MRSW Regular Boolean $\rightarrow$ MRSW Regular

8.2 Условия прогресса (progress conditions). Dependent blocking progress conditions. Примеры. Non-blocking progress conditions. Примеры. Dependent non-blocking progress conditions. Пример. Теорема о связи условий прогресса между собой.

Регистр (register): емкость (capacity), поддерживаемая конкурентность (concurrency), гарантии консистентности (consistency). Пространство регистров. Wait-free реализация объекта.

Теоремы о выразительной силе:

- MRSW Regular $\rightarrow$ SRSW Atomic
- SRSW Atomic $\rightarrow$ MRSW Atomic

8.3 Условия прогресса (progress conditions). Dependent blocking progress conditions. Примеры. Non-blocking progress conditions. Примеры. Dependent non-blocking progress conditions. Пример. Теорема о связи условий прогресса между собой.

Регистр (register): емкость (capacity), поддерживаемая конкурентность (concurrency), гарантии консистентности (consistency). Пространство регистров. Wait-free реализация объекта.

Атомарный снимок состояния N регистров (atomic snapshot). Алгоритм **CleanCollect** и его недостатки, пример ABA проблемы. Алгоритм **SimpleSnapshot**, обоснование корректности и его основных свойств.

9 Лекция 9: introduction to atomics

Ключевые понятия: read-modify-write, get-and-add, compare-and-swap, spin lock, lock-free stack, ABA problem

9.1 Условия прогресса (progress conditions). Dependent blocking progress conditions. Примеры. Non-blocking progress conditions. Примеры. Dependent non-blocking progress conditions. Пример. Теорема о связи условий прогресса между собой.

Консенсус. Теорема о невозможности достичь N-поточкового консенсуса с использованием только атомарных регистров. Практические следствия. Consensus number.

Read-modify-write objects. Примеры.

Теорема: **compareAndSet** обладает ∞ consensus number. Доказательство и практические следствия.

9.2 Взаимное исключения с помощью циклов и read-modify-write операций. Test-and-set-lock. Test-and-test-and-set-lock. Exponential backoff.

Проблемы масштабирования многопоточных систем, вызванные работой алгоритмов когерентности кешей. Вычислительные системы, использующие шину данных (bus-based architectures). Репликация линий кеша (cache line replication). Когерентность кешей (cache coherence). Проблема массовой инвалидации данных (invalidation storm).

9.3 Условия прогресса (progress conditions). Dependent blocking progress conditions. Примеры. Non-blocking progress conditions. Примеры. Dependent non-blocking progress conditions. Пример. Теорема о связи условий прогресса между собой.

Lock-free реализация потокобезопасного LIFO контейнера. Переиспользование памяти и АВА проблема.

Проблемы масштабирования многопоточных систем, вызванные работой алгоритмов когерентности кешей. Вычислительные системы, использующие шину данных (bus-based architectures). Репликация линий кеша (cache line replication). Когерентность кешей (cache coherence). Проблема массовой инвалидации данных (invalidation storm).

10 Лекция 10: cache coherency

Ключевые понятия: cache memory hierarchy, cache coherency protocol, store-buffer, load-buffer, invalidate-queue, memory barrier, hardware memory model, weak memory model, litmus tests

10.1 Наивные способы синхронизации глобальной памяти. Кеш-линия (cache line). Истинное и ложное разделение данных (true sharing, false sharing). Когерентность (coherence). Ослабленные требования к консистентности независимых ячеек памяти.

Построение эффективной подсистемы памяти: кеширование, шаблон "репликация", протокол обмена сообщениями. MESI протокол и примеры его работы. Преимущества и недостатки.

10.2 Наивные способы синхронизации глобальной памяти. Кеш-линия (cache line). Истинное и ложное разделение данных (true sharing, false sharing). Когерентность (coherence). Ослабленные требования к консистентности независимых ячеек памяти.

MESI протокол: состояния и их семантика, виды сообщений и их классификация (синхронные/асинхронные). Буферизация записи (store buffering), требования корректности (store forwarding).

10.3 Наивные способы синхронизации глобальной памяти. Кеш-линия (cache line). Истинное и ложное разделение данных (true sharing, false sharing). Когерентность (coherence). Ослабленные требования к консистентности независимых ячеек памяти.

MESI протокол: состояния и их семантика, виды сообщений и их классификация (синхронные/асинхронные). Буферизация чтений (load buffering). Топология шины данных (interconnect topology).

10.4 Наивные способы синхронизации глобальной памяти. Кеш-линия (cache line). Истинное и ложное разделение данных (true sharing, false sharing). Когерентность (coherence). Ослабленные требования к консистентности независимых ячеек памяти.

Модель памяти процессора (hardware memory model). Слабая модель памяти (weak memory model). Барьеры памяти (memory barriers), их классификация. Преимущества и недостатки барьерного подхода к написанию корректных конкурентных программ.

11 Лекция 11: language memory model

Ключевые понятия: compiler optimizations, compiler barriers, language memory model, strict consistency, threads cannot be implemented as a library, visibility, volatile

11.1 Оптимизации компилятора: влияние на наблюдаемое поведение однопоточной и многопоточной программы. Видимость операций с памятью (visibility). Барьеры для оптимизаций (compiler barriers), их таксономия. Необходимость использовать специальные языковые конструкции для "борьбы" с оптимизациями.

11.2 Модель памяти языка программирования (language memory model). Мотивация, цели, сложности при написании.

Неизменяемость (immutability). Декларативный подход к описанию параллельных вычислений. Строгая консистентность (strict consistency) с помощью взаимного исключения. Строгая консистентность (strict consistency) с помощью однопоточного исполнения. Реализация потоков на уровне библиотеки языка: преимущества и недостатки. Формальная модель памяти языка программирования. Альтернативы.

11.3 Модель памяти языка программирования (language memory model). Мотивация, цели, сложности при написании.

Видимость операций с памятью (visibility). Load-acquire/store-release подход к описанию консистентности. Ключевое слово `volatile`. Рекомендации к использованию.

12 Лекция 12: advanced locking

Ключевые понятия: Anderson Queue Lock, CLH, MCS, check-then-act, spin-then-park

12.1 Взаимное исключение. Критическая секция (mutex). Алгоритм взаимного исключения для двух потоков, основанный на флагах. Доказательство свойства mutual exclusion. Доказательство свойства deadlock freedom. Голодание (starvation). Реентрабельность (reentrancy). Admission policy, связь с честностью (fairness), наблюдаемым временем отклика (latency) и производительностью (throughput).

Наивные способы синхронизации глобальной памяти. Кеш-линия (cache line). Истинное и ложное разделение данных (true sharing, false sharing). Когерентность (coherence). Ослабленные требования к консистентности независимых ячеек памяти.

Test-and-test-and-set-lock. **Anderson Queue Lock**. Abortable locks: преимущества и сложности в реализации.

Проблема потерянного сигнала (lost signal), устаревания условия (predicate invalidation), внезапного пробуждения (spurious wakeup), грохочущего стада (thundering herd), инверсии приоритета (priority inversion), голодания (starvation), конвоя (lock convoy).

12.2 Взаимное исключение. Критическая секция (mutex). Алгоритм взаимного исключения для двух потоков, основанный на флагах. Доказательство свойства mutual exclusion. Доказательство свойства deadlock freedom. Голодание (starvation). Реентрабельность (reentrancy). Admission policy, связь с честностью (fairness), наблюдаемым временем отклика (latency) и производительностью (throughput).

Наивные способы синхронизации глобальной памяти. Кеш-линия (cache line). Истинное и ложное разделение данных (true sharing, false sharing). Когерентность (coherence). Ослабленные требования к консистентности независимых ячеек памяти.

Test-and-test-and-set-lock. **CLH Lock**. Abortable locks: преимущества и сложности в реализации.

Проблема потерянного сигнала (lost signal), устаревания условия (predicate invalidation), внезапного пробуждения (spurious wakeup), грохочущего стада (thundering herd), инверсии приоритета (priority inversion), голодания (starvation), конвоя (lock convoy).

- 12.3 Взаимное исключение. Критическая секция (mutex). Алгоритм взаимного исключения для двух потоков, основанный на флагах. Доказательство свойства mutual exclusion. Доказательство свойства deadlock freedom. Голодание (starvation). Реентрабельность (reentrancy). Admission policy, связь с честностью (fairness), наблюдаемым временем отклика (latency) и производительностью (throughput).

Наивные способы синхронизации глобальной памяти. Кеш-линия (cache line). Истинное и ложное разделение данных (true sharing, false sharing). Когерентность (coherence). Ослабленные требования к консистентности независимых ячеек памяти.

Test-and-test-and-set-lock. **MCS lock**. Abortable locks: преимущества и сложности в реализации.

Проблема потерянного сигнала (lost signal), устаревания условия (predicate invalidation), внезапного пробуждения (spurious wakeup), грохочущего стада (thundering herd), инверсии приоритета (priority inversion), голодания (starvation), конвоя (lock convoy).

- 12.4 Взаимное исключение. Критическая секция (mutex). Алгоритм взаимного исключения для двух потоков, основанный на флагах. Доказательство свойства mutual exclusion. Доказательство свойства deadlock freedom. Голодание (starvation). Реентрабельность (reentrancy). Admission policy, связь с честностью (fairness), наблюдаемым временем отклика (latency) и производительностью (throughput).

Наивные способы синхронизации глобальной памяти. Кеш-линия (cache line). Истинное и ложное разделение данных (true sharing, false sharing). Когерентность (coherence). Ослабленные требования к консистентности независимых ячеек памяти.

Test-and-test-and-set-lock. Exponential backoff. LockSupport.park, LockSupport.unpark. **Spin-then-park**. Check-then-act.

Проблема потерянного сигнала (lost signal), устаревания условия (predicate invalidation), внезапного пробуждения (spurious wakeup), грохочущего стада (thundering herd), инверсии приоритета (priority inversion), голодания (starvation), конвоя (lock convoy).